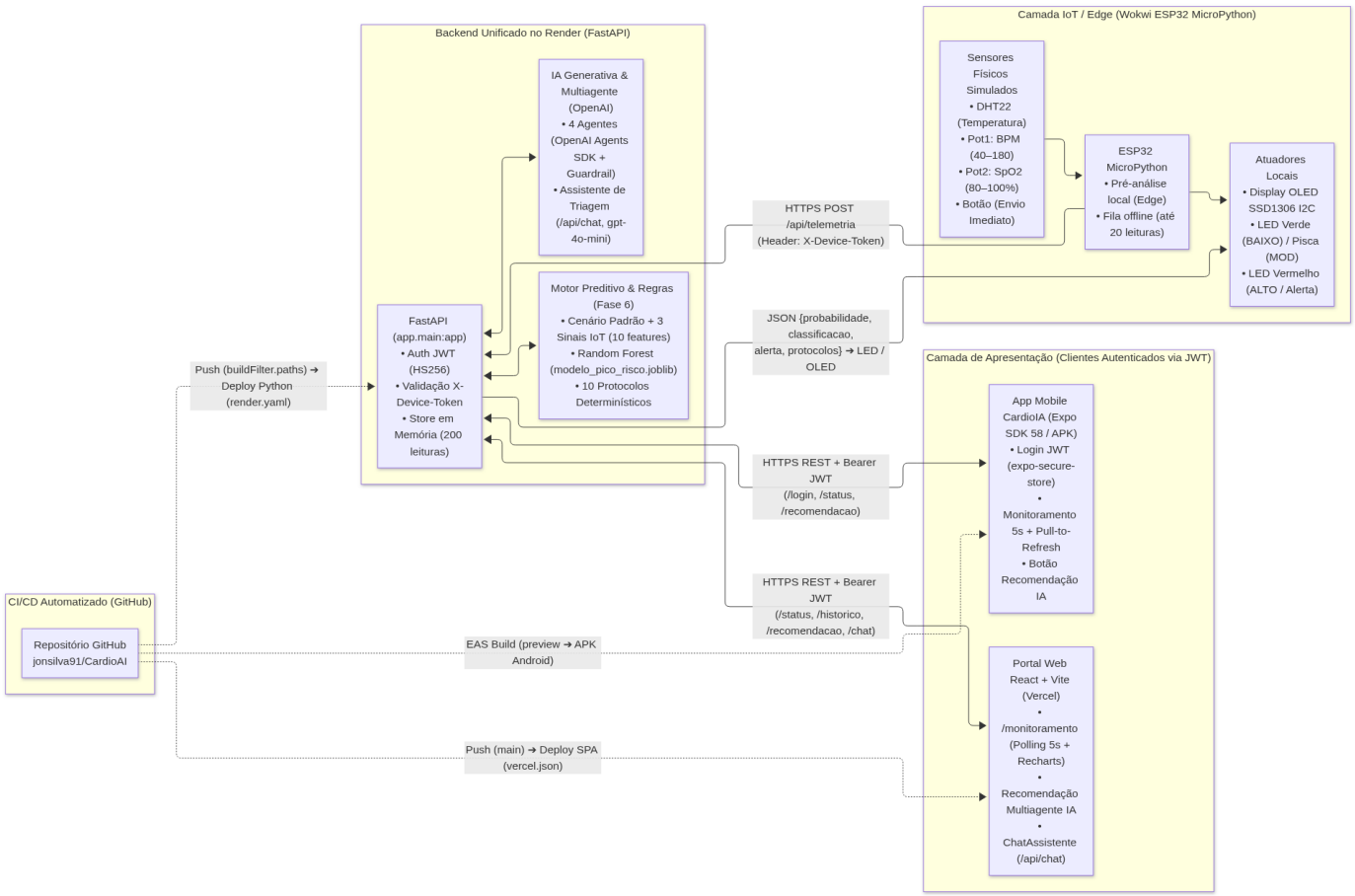
Relatório Técnico — Fase 7: Plataforma Integrada CardioAI

FIAP — Faculdade de Informática e Administração Paulista | **Integrantes:** Jonas Luis da Silva (RM561465) & Edson Henrique Felix Batista (RM566321)
Portal (Vercel): https://cardio-ai-five.vercel.app | **Backend (Render):** https://cardioai-fase07-backend.onrender.com (/docs, /health)
Build APK (EAS): https://expo.dev/accounts/jonsilva91/projects/mobile-cardioia/builds/be64508a-ebc2-4fb2-9ab3-93a89bb8b267
Projeto Wokwi (MicroPython): https://wokwi.com/projects/480500031706588161 | **Login Demo (Sintético):** demo@cardioia.com / CardioIA@2026 (Paciente: PAC-001)

1. Objetivo da Fase 7

Consolidar os módulos desenvolvidos nas fases anteriores do ecossistema **CardioAI** em uma plataforma integrada de ponta a ponta em nuvem, conectando um dispositivo de borda IoT (**ESP32 em MicroPython no Wokwi**), um **Backend FastAPI no Render** que importa diretamente o modelo preditivo Random Forest (99,9% de acurácia), os protocolos clínicos determinísticos e o sistema multiagente (OpenAI Agents SDK) da Fase 6, um assistente conversacional de triagem (gpt-4o-mini), o **Portal Web React/Vite na Vercel** e o **App Mobile Android (APK via EAS Build)**.

2. Diagrama de Arquitetura Final



3. Fluxo dos Dados Ponta a Ponta

• **1. Coleta e Pré-Análise Edge (Wokwi ESP32):** A cada 5 s (ou ao pressionar o botão no GPIO 15), o ESP32 lê a temperatura no DHT22 (GPIO 4), o BPM (40–180) no potenciômetro 1 (GPIO 34) e a SpO2 (80–100%) no potenciômetro 2 (GPIO 35), executa pré-análise local (BPM > 120, SpO2 < 92, temp > 38 °C) e atualiza o OLED SSD1306 I2C.
• **2. Ingestão Segura (POST /api/telemetria):** Envia JSON com paciente_id='PAC-001' e header X-Device-Token.
• **3. Inferência ML + Protocolos (Fase 6):** O backend completa as 3 leituras do sensor com as 7 variáveis do CENARIO_PADRAO (totalizando as 10 features), executa src.predictor.prever_risco e src.protocols.consultar_protocolos, guarda nas últimas 200 leituras em memória e devolve {probabilidade, classificacao, alerta, protocolos}.
• **4. Retorno ao Dispositivo IoT:** O ESP32 exibe 'Risco 0.xx ALTO' no OLED e controla os LEDs: vermelho (GPIO 5) se ALTO/alerta, verde (GPIO 2) se BAIXO e pisca se MODERADO.
• **5. Consumo Web/Mobile e IA Sob Demanda:** Portal (Vercel) e App (APK) autenticam via JWT (POST /api/auth/login), fazem polling a cada 5 s em /status e /historico, e acionam sob demanda os 4 agentes em POST /api/pacientes/PAC-001/recomendacao e o assistente de triagem em POST /api/chat.

4. Decisões Arquiteturais

• **HTTP REST (urequests) em vez de MQTT:** Permite que a própria resposta HTTP síncrona de POST /api/telemetria retorne o score calculado pelo modelo Random Forest na nuvem diretamente para o ESP32 comandar os LEDs e o display OLED, eliminando a dependência de brokers MQTT públicos intermediários.
• **Polling de 5 s nos Clientes:** Sincronizado ao ciclo de 5 s do ESP32, exibindo idade da leitura em segundos ('Ao vivo / sem dados há X s') e reconectando automaticamente após hibernação do container.
• **LLM e Agentes Apenas Sob Demanda (Controle de Custos):** A telemetria contínua a cada 5 s utiliza apenas o Random Forest (inferência local em microssegundos, custo zero). O pipeline de 4 agentes (OpenAI Agents SDK) e o chat (gpt-4o-mini) só rodam quando solicitados pelo usuário, com *rate-limit* de 1 chamada a cada 30 s por paciente (HTTP 429).
• **Sensores + Cenário Padrão ➔ 10 Features:** Combina fc_media, temperatura e spo2_media enviados pelo ESP32 com o baseline hospitalar (idade_media=62, carga_sistema=60, tempo_espera=40, pa_sistolica=120, densidade_eventos=3, eventos_24h=2, disponibilidade_recursos=60), reutilizando o modelo da Fase 6 sem cópia de código.
• **Edge com Pré-Análise Local e Fila de 20 Leituras:** Se o backend estiver acordando, o ESP32 sinaliza alertas locais na borda e armazena até 20 leituras em fila FIFO em RAM para envio posterior.

5. Deploy em Nuvem e CI/CD

• **Vercel (Portal React/Vite):** Deploy automático a cada push no GitHub. O arquivo apps/portal-cardioia/vercel.json aplica rewrite SPA de todas as rotas para /index.html e consome VITE_API_URL.
• **Render (Backend FastAPI):** Definido via Blueprint em render.yaml na raiz (plano Free, healthCheckPath: /health) com buildFilter.paths restrito ao backend da Fase 7, código/modelo da Fase 6 e render.yaml.
• **Expo EAS Build (APK Android):** Atualizado do SDK 49 para o **Expo SDK 58** (20/20 checks no expo-doctor), android.package: 'br.com.fiap.cardioia', versionCode: 1 e perfil preview gerando APK instalável.

6. Segurança, Variáveis de Ambiente e LGPD

• **Zero Segredos no Código:** JWT_SECRET, DEVICE_TOKEN, DEMO_EMAIL, DEMO_PASSWORD e OPENAI_API_KEY residem apenas nas variáveis de ambiente. O backend falha no startup (RuntimeError) se alguma variável obrigatória estiver ausente. GET /health informa apenas um booleano (openai_api_key: true/false).
• **Autenticação e CORS:** Rotas clínicas exigem Authorization: Bearer <JWT> (salvo no expo-secure-store no mobile); a ingestão IoT exige X-Device-Token validado com hmac.compare_digest; o CORS aceita apenas ALLOWED_ORIGINS.
• **LGPD e Ética Médica:** Uso exclusivo de dados sintéticos (PAC-001) e disclaimer fixo *'Não substitui avaliação médica.'*.

7. Conversão C++ (Fase 3) ➔ MicroPython (Fase 7)

Camada / Recurso	Fase 3 — C++ (sketch.ino)	Fase 7 — MicroPython (iot/main.py)
Transporte de Rede	PubSubClient / MQTT (HiveMQ)	urequests.post / HTTPS REST (X-Device-Token)
Sensor Temperatura	#include <DHT.h> / dht.readTemperature()	import dht / dht.DHT22(Pin(4)).temperature()
Leitura Analógica	analogRead(34) + map(...) (BPM)	ADC(Pin(34/35)).read() (BPM 40–180 e SpO2 80–100%)
GPIO Digital	digitalWrite / digitalRead	Pin.value(1/0) / Pin(15, Pin.IN, Pin.PULL_UP)
Saída Local Edge	Monitor Serial CSV (Serial.printf)	Display **OLED SSD1306 I2C** (ssd1306.py) + LEDs Verde/Vermelho

8. Resultados Observados em Produção

Cenário (PAC-001)	Leituras Enviadas	Risco Predito	Protocolos Acionados e Comportamento Observado
1. Taquicardia com SpO2 100%	BPM = 180 SpO2 = 100% Temp = 36.6 °C	**0.18 (18%)** **BAIXO**	1 protocolo individual (Se FC > 120 → iniciar monitoramento intensivo). Com SpO2 em 100% e cenário basal moderado, o Random Forest classifica 18% (BAIXO).
2. Descompensação Combinada	BPM = 180 SpO2 = 80% Temp = 36.6 °C	**0.86 (86%)** **ALTO**	**3 protocolos acionados**: (1) SPO2 < 88% → suporte respiratório; (2) FC > 120 → monitoramento intensivo; (3) risco alto → estado de emergência. OLED mostra Risco 0.86 ALTO e LED vermelho acende.
3. Paciente Basal Estável	BPM = 75 SpO2 = 98% Temp = 36.5 °C	**0.00 (0%)** **BAIXO**	0 protocolos acionados. OLED exibe Risco 0.00 BAIXO e LED verde permanece aceso.

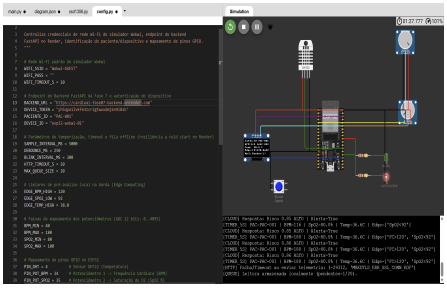
9. Limitações Conhecidas e Mitigações

• **Cold Start no Render Free (~50 s):** O serviço hiberna após inatividade; tratado com aviso *'Servidor acordando, aguarde'* no portal e fila no ESP32.
• **Histórico em Memória (200 leituras/paciente):** Armazenado em deque(maxlen=200), volátil em redeploys.
• **Quedas Ocasionais de TLS no Wokwi:** Contornadas pela fila local de até 20 leituras e reconexão automática no MicroPython.
• **CNN de ECG (Fase 4) Fora do Deploy:** TensorFlow removido do backend integrado para caber nos 512 MB de RAM do Render Free.
• **Dados Sintéticos e Emulação Android:** Dados 100% sintéticos. A validação do APK foi realizada no emulador **Android Pixel 7 / Android 13 via Appetize.io** (pois o integrante utiliza iOS), além da validação do build no EAS.

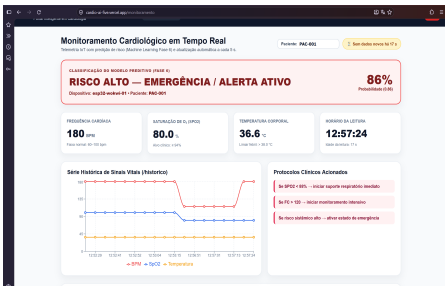
10. Próximos Passos

• Persistência em **PostgreSQL/TimescaleDB** e cache **Redis** para múltiplos workers; streaming via **WebSockets/SSE**; conversão da CNN da Fase 4 para **ONNX Runtime**; e prototipagem física com sensores biomédicos **MAX30102** e **DS18B20**.

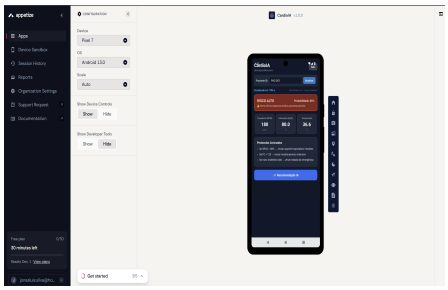
11. Evidências de Produção (Wokwi, Portal Vercel e APK no Appetize.io)



Wokwi ESP32 MicroPython (wokwi.png)



Portal Vercel (/monitoramento) (portal_monitoramento.png)



APK Android 13 (Appetize.io) (apk_appetize.png)